ARDUINO ROBOTICS & FLOOD-SAFE SMART HOUSE

Automatic House Lifting System Using Arduino UNO

A complete interactive STEM robotics and embedded systems curriculum covering microcontrollers, closed-loop servo actuation, L298N H-bridge motor drivers, ultrasonic time-of-flight acoustic ranging, precipitation telemetry, and an automated mechatronic flood protection architecture.

AUTHOR & PROJECT DESIGNER:

Toshit Kumar

Lead Robotics Engineer, Instructional Designer & Full-Stack Developer

STEM Robotics Education & Mechatronic Systems Architecture

Document Version: 2.4 | Academic & Laboratory Edition

Table of Contents

Section	Curriculum Topic & Engineering Specification	Page
01	Module 01: Introduction to Arduino UNO Architecture & First Sketch	3
02	Module 02: Servo Motor (SG90) Closed-Loop PWM Actuation	4
03	Module 03: Motor Driver (L298N Dual H-Bridge) & High-Torque Lifting	5
04	Module 04: HC-SR04 Ultrasonic Acoustic Time-of-Flight Telemetry	6
05	Module 05: Rain Sensor Module (FC-37) & Corrosion Prevention	7
06	Capstone Project: Flood-Safe Smart House Overview & Safety Rules	8
07	Capstone Project: Bill of Materials & Complete Connection Table	9
08	Capstone Project: Full Production Arduino Source Code	10
09	Scenario-Based Engineering Challenges & Decision Matrix	11
10	Comprehensive Assessment Key & Troubleshooting Guide	12

Created by Toshit Kumar

Module 01: Introduction to Arduino UNO

Microcontroller Fundamentals, Architecture, and First Sketch

What is Arduino & Microcontrollers?

Arduino is an open-source electronics platform based on easy-to-use hardware and software. At its heart lies a microcontroller—a compact integrated circuit designed to govern a specific operation in an embedded system.

Unlike a microprocessor (found in PCs and smartphones) which requires external RAM, ROM, and I/O peripherals to function, a microcontroller integrates the CPU core, volatile memory (SRAM), non-volatile program memory (Flash), rewritable storage (EEPROM), and programmable input/output peripherals onto a single monolithic silicon die.

Created by Toshit Kumar

Arduino UNO Hardware Architecture

The Arduino UNO R3 is built around the Microchip ATmega328P. Key hardware specifications include:
- Clock Speed: 16 MHz quartz crystal oscillator
- Flash Memory: 32 KB (0.5 KB occupied by bootloader)
- SRAM: 2 KB (volatile runtime variables)
- EEPROM: 1 KB (non-volatile persistent storage)
- Operating Voltage: 5V DC
- Recommended Input Voltage (VIN / Barrel Jack): 7V - 12V DC
- Absolute Maximum VIN: 6V - 20V (beyond 12V causes excessive linear regulator heating)
- DC Current per I/O Pin: 20 mA recommended (40 mA absolute maximum)
- DC Current for 3.3V Pin: 50 mA maximum

Pinout Taxonomy & Pin Categorization

The Arduino UNO exposes 28 header pins divided into specific functional groups:

1. Digital I/O Pins (D0 to D13):
   Configured via pinMode(pin, INPUT/OUTPUT). Logic HIGH represents +5V; logic LOW represents 0V.
   - D0 (RX) & D1 (TX): Hardware UART Serial communication. Leave disconnected during USB sketch upload.
   - D13: Connected to the onboard surface-mount LED through an internal buffer.

2. Pulse Width Modulation (PWM) Pins (~3, ~5, ~6, ~9, ~10, ~11):
   Indicated with a tilde (~) mark. Provide 8-bit pseudo-analog output (0 to 255) using analogWrite() at ~490 Hz (pins 5 and 6 run at ~980 Hz).

3. Analog Input Pins (A0 to A5):
   Read varying analog voltages (0V to 5V) using a 10-bit Successive Approximation Analog-to-Digital Converter (ADC). Resolution: 1024 discrete steps (0 to 1023), equaling ~4.88 mV per unit step. Can also serve as digital I/O pins (D14 to D19).

4. Power and Ground Pins:
   - VIN: Raw voltage input when using an external battery (7-12V).
   - 5V: Regulated +5V output from USB or onboard NCP1117 regulator.
   - 3.3V: Regulated +3.3V supply (LP2985-33) for low-voltage sensors.
   - GND: Ground reference (0V). All circuits must share a common ground.
   - AREF: Analog Reference voltage for the internal ADC.

The Arduino Execution Lifecycle: setup() & loop()

Every valid Arduino sketch contains two core functions:

```
void setup() {
  // Invoked exactly once when the board powers on or resets.
  // Used for initializing pin modes, starting Serial, configuring libraries.
}

void loop() {
  // Executes continuously in an infinite while(true) cycle.
  // Houses active sensor polling, mathematical computations, and actuator commands.
}
```

Pin / Terminal	Signal Type	Functional Engineering Description
D0 (RX)	Digital / UART	Serial Receive; connect to external TX. Used by USB bridge.
D1 (TX)	Digital / UART	Serial Transmit; connect to external RX. Used by USB bridge.
~D3, ~D5, ~D6, ~D9, ~D10, ~D11	Digital / PWM	8-bit Pulse Width Modulation output for motor speed and LED dimming.
D13	Digital	Standard Digital I/O tied to the onboard test LED.
A0 - A5	Analog Input	10-bit ADC inputs (0-1023) measuring 0V to 5V DC.
5V & 3.3V	Power Output	Regulated voltage rails for powering ICs and sensor modules.
GND (3 pins)	Power Ground	Common 0V reference potential for all connected electronics.
VIN	Power Input	Unregulated 7-12V DC input fed into the onboard 5V linear regulator.

Created by Toshit Kumar

Module 02: Servo Motor (SG90 / MG995)

Closed-Loop Angular Control, PWM Modulation, and Auxiliary Latch Mechanisms

Internal Anatomy of a RC Servo Motor

A servomotor is an integrated closed-loop electromechanical actuator comprising four essential components:
1. DC Motor: High-speed, low-torque prime mover.
2. Reduction Gearbox: Plastic (SG90) or brass/steel (MG995/MG996R) gear train that reduces rotational velocity while exponentially increasing torque.
3. Feedback Potentiometer: Mechanically coupled to the final output shaft, converting angular position into a proportional analog voltage.
4. Error Amplifier & Controller Board: Compares the target pulse position against the potentiometer feedback voltage. If an error exists, the H-bridge applies bidirectional drive current until shaft angle matches target position.

The 50 Hz PWM Pulse Protocol

Unlike motor speed PWM which varies duty cycle across arbitrary frequencies, standard analog RC servos require a strict 50 Hz frame frequency (one control pulse every 20 milliseconds / 20,000 µs):

- Neutral Position (90°): Pulse width of exactly 1.5 milliseconds (1500 µs).
- Full Minimum (0°): Pulse width of approximately 1.0 milliseconds (1000 µs, often calibrated to 544 µs in Arduino).
- Full Maximum (180°): Pulse width of approximately 2.0 milliseconds (2000 µs, often up to 2400 µs).

If the pulse stream ceases, the servo unpowers and offers no holding torque against external mechanical load.

Crucial Engineering Rule: Power Isolation

NEVER power a high-torque servo or multiple micro-servos directly from the Arduino UNO 5V header pin when driving mechanical load.

A micro SG90 draws 10-20 mA at idle, but its stall current spikes to 650-800 mA during sudden rotational acceleration. An MG995 draws up to 1.5A to 2.5A.
Attempting to draw this current from the Arduino 5V regulator causes severe voltage droop, tripping the microcontroller's internal Brown-Out Detector (BOD) and causing continuous, unpredictable MCU reset loops.

Always use an external 5V-6V power supply (e.g., 2A buck converter or battery pack) with common grounds tied together.

Pin / Terminal	Signal Type	Functional Engineering Description
Brown / Black	Ground (GND)	Negative return path. Must connect to BOTH external power ground and Arduino GND.
Red	Power (+5V to +6V)	Positive DC supply. Connect to external regulated 5V power supply (+ rail).
Orange / Yellow	Signal (PWM)	Control pulse input. Connect to Arduino PWM digital pin (e.g., Pin 9 or 10).

Created by Toshit Kumar

Module 03: Motor Driver (L298N Dual H-Bridge)

High-Current Inductive Drive, H-Bridge Topologies, and Bidirectional Lifting

Why Do We Need a Motor Driver?

A digital pin on an Arduino UNO can safely source a maximum of 20 to 40 milliamperes (mA) at 5 volts. A standard DC geared motor (such as a 12V TT gearmotor or planetary gearmotor used for lifting platforms) draws between 300 mA under no-load conditions to over 2.5 A when stalled or lifting a load. Connecting a motor directly to an Arduino pin will instantly vaporize the output stage transistor of the ATmega328P.

Furthermore, motors are inductive loads. When current through an inductor is suddenly interrupted, magnetic field collapse generates a massive reverse voltage spike (back-EMF: V = -L * di/dt) that can reach tens or hundreds of volts, destroying unshielded digital electronics. A motor driver isolates the microcontroller, provides flyback diode protection, and switches heavy currents using an external power source.

The H-Bridge Topology & Truth Table

An H-Bridge consists of four solid-state switching elements (BJTs or MOSFETs) arranged in the shape of an 'H' around the motor:

- Forward Rotation: Q1 (top-left) and Q4 (bottom-right) closed. Current flows Left to Right through the motor.
- Reverse Rotation: Q2 (top-right) and Q3 (bottom-left) closed. Current flows Right to Left through the motor.
- High-Impedance Coast: All switches open. Motor freewheels.
- Dynamic Braking: Q3 and Q4 (or Q1 and Q2) closed. Both motor terminals shorted together, dissipating kinetic energy as reverse magnetic braking.
- Shoot-Through (Catastrophic Fault): Closing Q1 and Q3 simultaneously creates a direct short from +V to GND. The L298N internal logic prevents shoot-through by hardware interlocks.

L298N Terminal Configuration & Jumper Rules

1. 12V Power Terminal: External DC supply for motors (7V - 35V). For our lifting platform, a 12V 3A DC power source is connected here.
2. GND Terminal: Common ground reference. MUST be connected to BOTH the 12V external power supply ground and the Arduino UNO GND pin.
3. 5V Logic Terminal: Powers the internal 78M05 voltage regulator.
   - If 5V Jumper is ON: Onboard regulator converts external 12V down to 5V to power internal logic. Do NOT connect external 5V to this pin.
   - If 5V Jumper is REMOVED (External motor > 12V): Onboard regulator is disabled. You must supply +5V from Arduino 5V to this pin.
4. ENA / ENB (Enable Pins): Speed control. Removing the jumper cap exposes the pin for an Arduino PWM signal (0-255).
5. IN1, IN2 (Motor A) / IN3, IN4 (Motor B): Direction control logic pins.

Pin / Terminal	Signal Type	Functional Engineering Description
12V Terminal	Power Input	Connects to positive lead of external 12V battery/power supply.
GND Terminal	Common Ground	Connects to negative lead of external supply AND Arduino GND.
5V Terminal	Logic Supply	5V output when 5V jumper is fitted; 5V input if jumper removed.
ENA	Speed Control	PWM input (~Pin 5) controlling Motor A speed from 0 (halt) to 255 (full).
IN1 & IN2	Direction Logic	Digital inputs (e.g. Pin 3 & Pin 4) controlling Motor A rotation direction.
OUT1 & OUT2	Motor Output	Heavy-gauge screw terminals wired directly to the DC lifting motor.

Created by Toshit Kumar

Module 04: Ultrasonic Sensor (HC-SR04)

Time-of-Flight Acoustic Ranging and Flood Depth Telemetry

Piezoelectric Ultrasonic Transducers & Working Principle

The HC-SR04 ultrasonic sensor utilizes acoustic echolocation to calculate distance without physical contact. The module incorporates two aluminum cylindrical transducers:
1. Transmitter (T): Contains a piezoelectric crystal disc that vibrates at 40,000 Hertz (40 kHz) when stimulated by high-frequency electrical pulses, radiating a focused conical ultrasonic beam into the air.
2. Receiver (R): Contains an identical crystal tuned to 40 kHz. When incoming reflected sound waves strike the diaphragm, mechanical stress generates a minute alternating electrical charge, which is amplified and conditioned by onboard LM324 operational amplifiers.

The Time-of-Flight (ToF) Physical Formula

Distance calculation relies on the speed of sound propagating through air at standard sea-level atmospheric pressure and 20°C:
Speed of Sound (c) "H 343 meters/second = 0.0343 centimeters/microsecond.

Because the acoustic burst travels outward from the transmitter to the reflective water surface and bounces all the way back to the receiver, the total distance traveled is twice the distance to the target:

Distance = (Time Elapsed * Speed of Sound) / 2
Distance (cm) = (Echo Time in µs * 0.0343) / 2
Simplified: Distance (cm) = Echo Time in µs / 58.3

Temperature Compensation Consideration:
Sound velocity varies with ambient air temperature: c(T) = 331.3 + (0.606 * T) m/s. In flood conditions where air humidity and temperature fluctuate, incorporating a DHT11 or DHT22 temperature sensor refines distance calculations to millimeter accuracy.

Water Level Monitoring Strategy in Flood Mitigation

When measuring floodwater depth, the ultrasonic sensor is mounted overhead facing downward into the retention basin or foundation channel:
- Basin Depth (Baseline Calibration): e.g., 50 cm from sensor to dry floor.
- Measured Ultrasonic Distance: The distance from the sensor down to the water surface.
- Calculated Water Depth: Flood Level = Baseline Calibration - Measured Distance.

As floodwater rises, the measured distance decreases.
Example: If the baseline is 50 cm and the sensor reads 15 cm, the floodwater depth is exactly 35 cm. If the critical threshold is set to 20 cm depth, an emergency lifting command is triggered immediately.

Pin / Terminal	Signal Type	Functional Engineering Description
VCC	Power Supply (+5V)	Regulated +5V DC power from Arduino 5V rail (draws ~15mA).
Trig (Trigger)	Digital Input	Microcontroller sends a 10-microsecond HIGH pulse to initiate ultrasonic sonic burst.
Echo	Digital Output	Module drives pin HIGH for duration equal to acoustic round-trip flight time.
GND	Power Ground	0V ground return reference connected to Arduino GND.

Created by Toshit Kumar

Module 05: Rain Sensor Module (FC-37 / YL-83)

Resistive Trace Detection, LM393 Comparator, and Precipitation Telemetry

Electrolytic Resistance Principle of Rain Sensing

The rain sensor system consists of two primary hardware elements:
1. Sensing Pad: A printed circuit board with interleaved, interdigitated copper/nickel conductive tracks. When dry, air acts as an electrical insulator, resulting in near-infinite resistance (several megaohms). When precipitation falls, water droplets contain dissolved minerals and ions that act as an electrolyte, creating conductive bridges between adjacent tracks and dramatically lowering electrical resistance.
2. Signal Conditioning Module: Features an LM393 dual differential voltage comparator, a sensitivity trim potentiometer, status LEDs, and output headers.

Understanding AO (Analog) and DO (Digital) Outputs

The sensor board provides two distinct output channels:

1. Analog Output (AO):
Provides an analog voltage proportional to the electrical resistance across the sensor tracks. In an Arduino with a 5V reference:
- Completely Dry: AO reads 1000 - 1023 (~5.0V, high resistance).
- Light Drizzle / Condensation: AO reads 600 - 800 (~3.0V - 4.0V).
- Moderate Rainfall: AO reads 400 - 600 (~2.0V - 3.0V).
- Heavy Downpour / Submersion: AO reads < 350 (< 1.7V, high conductivity).

2. Digital Output (DO):
The onboard LM393 compares the sensor voltage with a reference voltage set by the multi-turn potentiometer.
- When moisture exceeds the potentiometer threshold, DO flips from HIGH (5V) to LOW (0V), triggering an active-low hardware interrupt or digital read.

Created by Toshit Kumar

Preventing Galvanic & Electrolytic Corrosion

A critical engineering challenge with resistive rain sensors is galvanic oxidation. If DC current continuously flows through the wet nickel tracks, electrolysis rapidly oxidizes the metal, causing traces to corrode and disintegrate within days or weeks of deployment.

Professional Solution:
Do not connect the rain sensor VCC pin permanently to the Arduino 5V power rail. Instead, connect VCC to a digital I/O pin (e.g., Pin 7). Set the pin HIGH only 10 milliseconds before taking an analog reading, sample the voltage, and immediately pull the pin LOW (0V). This eliminates continuous galvanic current, extending sensor lifespan by over 500x.

Pin / Terminal	Signal Type	Functional Engineering Description
VCC	Power Input	Connect to Arduino 5V (or digital control pin for corrosion-free pulsed sampling).
GND	Power Ground	Common 0V ground connection with Arduino.
AO (Analog Out)	Analog Signal	Connects to Arduino Analog Input (A0) for granular 0-1023 precipitation intensity.
DO (Digital Out)	Digital Signal	Connects to Arduino Digital Input (D2) for binary threshold trigger (LOW on rain).

CAPSTONE PROJECT: FLOOD-SAFE SMART HOUSE

Automatic House Lifting System Using Arduino UNO | Designed by Toshit Kumar

Monsoon floods, rapid sea-level surges, and flash overflowing of river catchments inundate residential communities worldwide, causing catastrophic infrastructural destruction and loss of life. The Flood-Safe Smart House project engineers an automated mechatronic lifting structure. Combining dual-stage early precipitation telemetry (Rain Sensor) with real-time acoustic water depth profiling (HC-SR04 Ultrasonic), an Arduino UNO orchestrates a high-torque L298N lead-screw platform lift, audio-visual alarm annunciators, mechanical end-stop safety interlocks, and an auxiliary servo stabilizing gate lock.

Mandatory Safety Engineering Rules:

1. NEVER rely purely on software time delays (e.g. delay(5000)) to halt lifting or lowering. Mechanical limit switches or optical end-stops at top and bottom travel are MANDATORY fail-safe mechanisms.

2. ALWAYS establish a common ground (GND) across the external 12V motor battery and the Arduino UNO board. Lack of common ground causes floating gate potentials, erratic motor jitter, and lost logic signals.

3. DO NOT power the high-torque L298N DC motor or the SG90 servo directly from the Arduino 5V regulator pin. Motors must be powered from a dedicated 12V 3A DC power source.

4. Ensure the L298N heat sink has sufficient convection cooling. Stalled motors generate high thermal dissipation (up to 25W), which will trip thermal shutdown without proper ventilation.

5. EDUCATIONAL DISCLAIMER: This prototype is strictly an instructional engineering demonstration for robotics students. Real-world civil architectural house lifting requires certified hydraulic jacks, seismic dampening, structural beam engineering, and professional building code compliance.

Created by Toshit Kumar

Bill of Materials (BOM) & Hardware Specifications

Component	Qty	Engineering Role in Flood-Safe House
Arduino UNO R3	1	Central microcontroller orchestration, telemetry, state machine, and actuator signals.
L298N Dual H-Bridge Motor Driver	1	Provides high-current 12V drive for the DC vertical lifting motor with direction and PWM speed control.
12V High-Torque DC Geared Motor (50-100 RPM)	1	Prime mover driving the rack-and-pinion or threaded lead-screw lifting jack.
SG90 Micro Servo Motor	1	Controls the automated safety lock pin and flood barrier gate.
HC-SR04 Ultrasonic Sensor Module	1	Mounts overhead to measure real-time distance down to the rising floodwater surface.
FC-37 / YL-83 Rain Sensor Module	1	Detects early precipitation onset and classifies storm intensity (light, moderate, heavy).
Micro Limit Switches (Normally Closed / Open)	2	Hardware end-stops triggering upper travel limit and base datum foundation arrival.
Active Buzzer Module (5V)	1	Emits 85dB acoustic warning alarm to alert inhabitants of immediate flood risk.
LED Indicators (Red, Green, Resistors	3	Visual state annunciators (Red: Danger/Lift, Green: Safe Hold, Blue: Lowering).
12V 3A DC Power Supply / LiPo Battery Pack	1	Dedicated heavy-current power supply for the DC lifting motor.
Solderless Breadboard & Premium Jumper Wires	1	Distributes power rails and connects modular sub-circuits cleanly.
Scale House & Scissor / Rack-and-Pinion Lift Chassis	1	Mechanical lifting platform structure constructed from acrylic, balsa wood, or 3D print.

Created by Toshit Kumar

Complete Circuit Connection Matrix

Hardware Module	Module Pin	Arduino UNO Pin	Operational Function / Protection
HC-SR04 Ultrasonic	VCC	5V	Powers internal 40kHz oscillator
HC-SR04 Ultrasonic	Trig	Digital Pin 9	Sends 10µs trigger pulse
HC-SR04 Ultrasonic	Echo	Digital Pin 10	Receives ToF pulse width
HC-SR04 Ultrasonic	GND	GND	Common system ground
Rain Sensor Module	VCC	Digital Pin 7	Pulsed power pin for anti-corrosion
Rain Sensor Module	AO	Analog Pin A0	Moisture resistance (0-1023)
Rain Sensor Module	GND	GND	Common system ground
L298N Motor Driver	ENA	Digital Pin 5 (~PWM)	Speed control (0-255 duty cycle)
L298N Motor Driver	IN1	Digital Pin 3	Direction control pin 1 (Lift)
L298N Motor Driver	IN2	Digital Pin 4	Direction control pin 2 (Lower)
L298N Motor Driver	12V Terminal	External 12V (+)	DO NOT wire to Arduino 5V
L298N Motor Driver	GND Terminal	GND & Ext 12V (-)	Mandatory common ground
SG90 Servo Motor	Signal (Yellow)	Digital Pin 6 (~PWM)	Auxiliary safety latch control

Hardware Module	Module Pin	Arduino UNO Pin	Operational Function / Protection
SG90 Servo Motor	VCC (Red)	External 5V (+)	Do not power from Arduino 5V pin under load
SG90 Servo Motor	GND (Brown)	GND	Common system ground
Upper Limit Switch	Signal Out	Digital Pin 2 (INT0)	Triggers when platform hits upper ceiling
Lower Limit Switch	Signal Out	Digital Pin 8	Triggers when platform arrives at ground
Red LED (Danger/Lift)	Anode (+ via 220	Digital Pin 11	Active when flood threshold breached
Green LED (Safe Hold)	Anode (+ via 220	Digital Pin 12	Active during normal ground or locked high
Blue LED (Lowering)	Anode (+ via 220	Digital Pin 13	Active during descent operation
Active Buzzer	Anode (+)	Digital Pin A1	Generates audible pulsed alert tones

Created by Toshit Kumar

Full Arduino Production Source Code

```
/*
  ================================================================================
  PROJECT: FLOOD-SAFE SMART HOUSE: AUTOMATIC HOUSE LIFTING SYSTEM
  DEVELOPER: Toshit Kumar
  PLATFORM: Arduino UNO R3
  VERSION: 2.4 (Production Grade Educational Mechatronic Implementation)
  ================================================================================
*/

#include <Servo.h>

// --- PIN ASSIGNMENTS ---
// Ultrasonic Sensor (HC-SR04)
const int PIN_TRIG = 9;
const int PIN_ECHO = 10;

// Rain Sensor (FC-37)
const int PIN_RAIN_PWR = 7;
const int PIN_RAIN_AO  = A0;

// L298N Motor Driver (12V DC Lifting Motor)
const int PIN_ENA = 5;  // PWM Speed Control
const int PIN_IN1 = 3;  // Direction Logic 1 (Lift)
const int PIN_IN2 = 4;  // Direction Logic 2 (Lower)

// Auxiliary SG90 Servo Motor (Safety Lock / Barrier)
const int PIN_SERVO = 6;
Servo safetyLatchServo;

// Mechanical Limit Switches (End-Stops)
const int PIN_LIMIT_TOP    = 2; // Upper mechanical limit (Interrupt INT0)
const int PIN_LIMIT_BOTTOM = 8; // Lower mechanical datum limit

// Audio & Visual Annunciators
const int PIN_LED_RED   = 11; // Danger / Lifting
const int PIN_LED_GREEN = 12; // Safe Standby / Holding
const int PIN_LED_BLUE  = 13; // Lowering Active
const int PIN_BUZZER    = A1; // Acoustic Flood Alarm

// --- CALIBRATION & THRESHOLD CONSTANTS ---
const float SENSOR_DATUM_HEIGHT   = 40.0; // Distance from sensor to dry ground (cm)
const float FLOOD_TRIGGER_DEPTH   = 15.0; // Water depth in cm that triggers LIFT
const float FLOOD_SAFE_DEPTH      = 5.0;  // Water depth in cm safe for LOWERING
const int   RAIN_THRESHOLD        = 700;  // Rain AO threshold (lower = heavier rain)
const int   LIFT_SPEED            = 230;  // L298N PWM speed for upward lift (0-255)
const int   LOWER_SPEED           = 175;  // L298N PWM speed for controlled descent
const unsigned long DEBOUNCE_WAIT = 5000; // 5-second hysteresis debounce

// --- SYSTEM STATE MACHINE ---
enum SystemState {
  STATE_STANDBY_GROUND,
  STATE_FLOOD_ALERT,
  STATE_LIFTING,
  STATE_HOLDING_ELEVATED,
  STATE_LOWERING
};

SystemState currentState = STATE_STANDBY_GROUND;
unsigned long safeTimerStart = 0;
bool safeTimerRunning = false;

// --- PROTOTYPES ---
float getWaterDepthCm();
int getRainReading();
void setActuators(SystemState state);
void stopMotor();

void setup() {
  Serial.begin(9600);
  Serial.println("================================================");
  Serial.println(" FLOOD-SAFE SMART HOUSE INITIALIZING...");
```

Created by Toshit Kumar

```
  Serial.println(" Designed by Toshit Kumar | STEM Robotics Lab");
  Serial.println("=================================================");

  // Pin Configurations
  pinMode(PIN_TRIG, OUTPUT);
  pinMode(PIN_ECHO, INPUT);
  pinMode(PIN_RAIN_PWR, OUTPUT);
  digitalWrite(PIN_RAIN_PWR, LOW);

  pinMode(PIN_ENA, OUTPUT);
  pinMode(PIN_IN1, OUTPUT);
  pinMode(PIN_IN2, OUTPUT);

  pinMode(PIN_LIMIT_TOP, INPUT_PULLUP);
  pinMode(PIN_LIMIT_BOTTOM, INPUT_PULLUP);

  pinMode(PIN_LED_RED, OUTPUT);
  pinMode(PIN_LED_GREEN, OUTPUT);
  pinMode(PIN_LED_BLUE, OUTPUT);
  pinMode(PIN_BUZZER, OUTPUT);

  // Initialize Servo
  safetyLatchServo.attach(PIN_SERVO);
  safetyLatchServo.write(0); // 0 = Mechanical Foundation Lock Engaged

  // Stop Motor Initially
  stopMotor();

  // Indicate Ready State
  digitalWrite(PIN_LED_GREEN, HIGH);
  Serial.println("Status: System Ready. Standby at Foundation Level.");
}

void loop() {
  // 1. Measure Environmental Parameters
  float waterDepth = getWaterDepthCm();
  int rainValue    = getRainReading();

  // Read Safety Limit Switches (Active LOW with INPUT_PULLUP)
  bool hitTopLimit    = (digitalRead(PIN_LIMIT_TOP) == LOW);
  bool hitBottomLimit = (digitalRead(PIN_LIMIT_BOTTOM) == LOW);

  // Debug Telemetry
  Serial.print("[TELEMETRY] Water Depth: ");
  Serial.print(waterDepth);
  Serial.print(" cm | Rain AO: ");
  Serial.print(rainValue);
  Serial.print(" | State: ");

  // 2. State Machine Logic
  switch (currentState) {

    case STATE_STANDBY_GROUND:
      Serial.println("STANDBY_GROUND");
      digitalWrite(PIN_LED_GREEN, HIGH);
      digitalWrite(PIN_LED_RED, LOW);
      digitalWrite(PIN_LED_BLUE, LOW);
      digitalWrite(PIN_BUZZER, LOW);

      // Check if flood condition verified
      if (waterDepth >= FLOOD_TRIGGER_DEPTH || (rainValue < RAIN_THRESHOLD && waterDepth >=
(FLOOD_TRIGGER_DEPTH * 0.7))) {
        Serial.println(">>> WARNING: Flood danger detected! Transitioning to ALERT.");
        currentState = STATE_FLOOD_ALERT;
      }
      break;

    case STATE_FLOOD_ALERT:
      Serial.println("FLOOD_ALERT");
      digitalWrite(PIN_LED_RED, HIGH);
      digitalWrite(PIN_LED_GREEN, LOW);
      // Audible warning tone
      digitalWrite(PIN_BUZZER, HIGH);
```

Created by Toshit Kumar

```
    delay(200);
    digitalWrite(PIN_BUZZER, LOW);

    // Disengage foundation safety latch
    safetyLatchServo.write(90);
    delay(500); // Allow servo motion

    currentState = STATE_LIFTING;
    break;

  case STATE_LIFTING:
    Serial.println("LIFTING_ACTIVE");
    digitalWrite(PIN_LED_RED, HIGH);
    digitalWrite(PIN_LED_GREEN, LOW);
    digitalWrite(PIN_LED_BLUE, LOW);

    // Intermittent alert beep
    digitalWrite(PIN_BUZZER, (millis() % 500 < 100) ? HIGH : LOW);

    if (hitTopLimit) {
      Serial.println(">>> Upper Limit Switch Engaged! Stopping motor at summit.");
      stopMotor();
      // Engage elevated support lock
      safetyLatchServo.write(180);
      currentState = STATE_HOLDING_ELEVATED;
    } else {
      // Drive motor upwards
      digitalWrite(PIN_IN1, HIGH);
      digitalWrite(PIN_IN2, LOW);
      analogWrite(PIN_ENA, LIFT_SPEED);
    }
    break;

  case STATE_HOLDING_ELEVATED:
    Serial.println("HOLDING_ELEVATED");
    stopMotor();
    digitalWrite(PIN_LED_RED, LOW);
    digitalWrite(PIN_LED_GREEN, HIGH);
    digitalWrite(PIN_LED_BLUE, LOW);
    digitalWrite(PIN_BUZZER, LOW);

    // Check if flood water has receded safely
    if (waterDepth <= FLOOD_SAFE_DEPTH) {
      if (!safeTimerRunning) {
        safeTimerStart = millis();
        safeTimerRunning = true;
        Serial.println("Water receded below safe margin. Starting verification timer...");
      } else if (millis() - safeTimerStart >= DEBOUNCE_WAIT) {
        Serial.println("Flood subsided sustainably. Initiating descent sequence.");
        safeTimerRunning = false;
        // Retract support locks before lowering
        safetyLatchServo.write(90);
        delay(600);
        currentState = STATE_LOWERING;
      }
    } else {
      safeTimerRunning = false; // Reset timer if water fluctuates back up
    }
    break;

  case STATE_LOWERING:
    Serial.println("LOWERING_ACTIVE");
    digitalWrite(PIN_LED_RED, LOW);
    digitalWrite(PIN_LED_GREEN, LOW);
    digitalWrite(PIN_LED_BLUE, HIGH);
    digitalWrite(PIN_BUZZER, LOW);

    if (hitBottomLimit) {
      Serial.println(">>> Lower Datum Switch Engaged! Platform safely docked on ground.");
      stopMotor();
      // Engage ground foundation lock
      safetyLatchServo.write(0);
      currentState = STATE_STANDBY_GROUND;
```

Created by Toshit Kumar

```
      } else {
        // Drive motor downwards smoothly
        digitalWrite(PIN_IN1, LOW);
        digitalWrite(PIN_IN2, HIGH);
        analogWrite(PIN_ENA, LOWER_SPEED);
      }
      break;
  }

  delay(100); // 10Hz control loop
}

// --- HELPER ROUTINES ---

float getWaterDepthCm() {
  digitalWrite(PIN_TRIG, LOW);
  delayMicroseconds(2);
  digitalWrite(PIN_TRIG, HIGH);
  delayMicroseconds(10);
  digitalWrite(PIN_TRIG, LOW);

  long durationUs = pulseIn(PIN_ECHO, HIGH, 26000); // ~4.5m max
  if (durationUs == 0) return 0.0; // Assume dry if out of range

  float distanceToSurface = (durationUs * 0.0343) / 2.0;
  float depth = SENSOR_DATUM_HEIGHT - distanceToSurface;
  return (depth < 0.0) ? 0.0 : depth;
}

int getRainReading() {
  digitalWrite(PIN_RAIN_PWR, HIGH);
  delay(10);
  int val = analogRead(PIN_RAIN_AO);
  digitalWrite(PIN_RAIN_PWR, LOW);
  return val;
}

void stopMotor() {
  digitalWrite(PIN_IN1, LOW);
  digitalWrite(PIN_IN2, LOW);
  analogWrite(PIN_ENA, 0);
}
```

Created by Toshit Kumar

Scenario-Based Decision Exercises & Engineering Matrix

Scenario 01: Rain Starts but Water Level Remains Safe

Context: A heavy monsoon thunderstorm begins in the catchment zone. The rain sensor AO drops sharply to 320 (indicating intense downpour). However, the overhead HC-SR04 ultrasonic sensor reports a distance of 38 cm (indicating only 2 cm of ground moisture in the drainage gutter, well below the 15 cm danger threshold).

Optimal Protocol: Sound a pre-warning status, keep the house on the ground, and increase ultrasonic polling frequency to monitor depth

Rationale: Modern civil flood barriers implement multi-sensor redundancy. Rain sensors provide early weather awareness, but actual physical elevation should only occur when hydrological accumulation (water level depth) is verified by the acoustic or hydrostatic depth sensor.

Scenario 02: Water Level Suddenly Surges Past Critical Threshold

Context: A flash flood surge occurs following a riverbank breach. The ultrasonic sensor detects water rising rapidly from 4 cm to 18 cm within 20 seconds, exceeding the 15 cm critical threshold.

Optimal Protocol: Sound buzzer & illuminate Red LED -> Disengage servo safety locks -> Command L298N forward lifting -> Monitor uppe

Rationale: Sequential interlock control is fundamental to mechatronics. Actuators must follow a strict non-conflicting dependency graph: Annunciation -> Unlock -> Drive -> Limit Stop -> Lock.

Scenario 03: Ultrasonic Sensor Detects Erroneous Distance During Waves

Context: Violent wind causes turbulent waves in the flood basin. Over 5 seconds, the ultrasonic sensor returns: [14cm, 0cm, 15cm, 298cm, 14.5cm]. The 0cm and 298cm readings are acoustic reflection timeouts caused by wave tilt reflecting sound away from the receiver.

Optimal Protocol: Implement a median filter (e.g. sample 5 values, discard min/max outliers, sort and take median) before passing to the s

Rationale: Digital signal processing (DSP) techniques such as running median filters, moving averages, and range-clamping are mandatory when reading ultrasonic sensors over liquid surfaces subject to turbulence.

Scenario 04: House Reaches its Upper Lifting Limit

Context: The house has elevated 25 cm above datum. The platform chassis makes contact with the physical Upper Limit Switch, driving digital Pin 2 LOW. However, the ultrasonic sensor still detects 22 cm of water depth.

Optimal Protocol: Immediately deactivate motor drive (IN1=LOW, IN2=LOW, ENA=0), apply dynamic brake, and engage servo support pins

Rationale: Hardware limit switches provide definitive physical datum points. In industrial robotics, limit switch inputs are often routed directly to hardware interrupts (such as Arduino INT0 on Pin 2) to ensure sub-millisecond motor cutoff.

Scenario 05: Water Level Returns to Normal (Subsided)

Context: The storm has passed, flood gates downstream open, and water depth drops to 3 cm (below the 5 cm SAFE_THRESHOLD). The house is currently held at the upper summit.

Optimal Protocol: Start a hysteresis timer (e.g. 10 continuous seconds below safe margin) -> Retract servo lock pins -> Lower at moderate

Rationale: Hysteresis timing prevents 'chattering' in control systems. When transitioning between states across dynamic environmental variables (like flood recession), requiring sustained safe conditions prevents erratic actuator oscillation.

Created by Toshit Kumar

Troubleshooting & Diagnostics Reference

Fault Symptom	Probable Root Cause	Field Remedy & Rectification
Motor does not turn despite water exceeding flood threshold.	Missing common ground between 12V battery and Arduino, or 12V power supply switched off.	Verify with a multimeter that Arduino GND and L298N GND terminal share 0V continuity. Verify 12V supply is active.
Arduino resets continuously when motor or servo starts moving.	High stall current drawing down Arduino 5V rail; power supply brownout.	Disconnect servo and motor power from Arduino 5V. Use dedicated external 12V and 5V power rails.
Ultrasonic sensor reads erratic distances (e.g. 0 cm or jumps to 300 cm).	Surface water ripples scattering acoustic waves, or pulseIn timeout.	Mount a small acoustic stilling tube (PVC pipe with bottom hole) around the water sensor to prevent water surface agitation.
Motor continues driving after reaching maximum height, causing gear grind.	Upper limit switch failed to engage, bad wiring, or pull-up resistor omitted.	Test switch using Serial monitor. Ensure pinMode(PIN_LIMIT_TOP, INPUT_PULLUP) is configured and wiring makes reliable contact.
Rain sensor traces turn black and stop conducting within a week.	Galvanic corrosion caused by continuous DC current during wet exposure.	Switch sensor power dynamically using Arduino digital Pin 7 as implemented in our production code.

Created by Toshit Kumar

Curriculum Accreditation & Author Verification

This documentation is prepared as part of the STEM Robotics Learning Curriculum.

Author & Lead Instructional Designer: Toshit Kumar

Published for interactive classroom training, robotics laboratories, and self-paced study.